

LABORATORY PROJECT DOCUMENTATION



Authors:

Marcos Loro Carrasco. Participation: 50%

Ignacio Romero Torres. Participation: 50%

INDEX:

1. [Analysis of the requirements.](#)
2. [UML Diagrams.](#)
3. [User manual.](#)

Analysis of requirements:

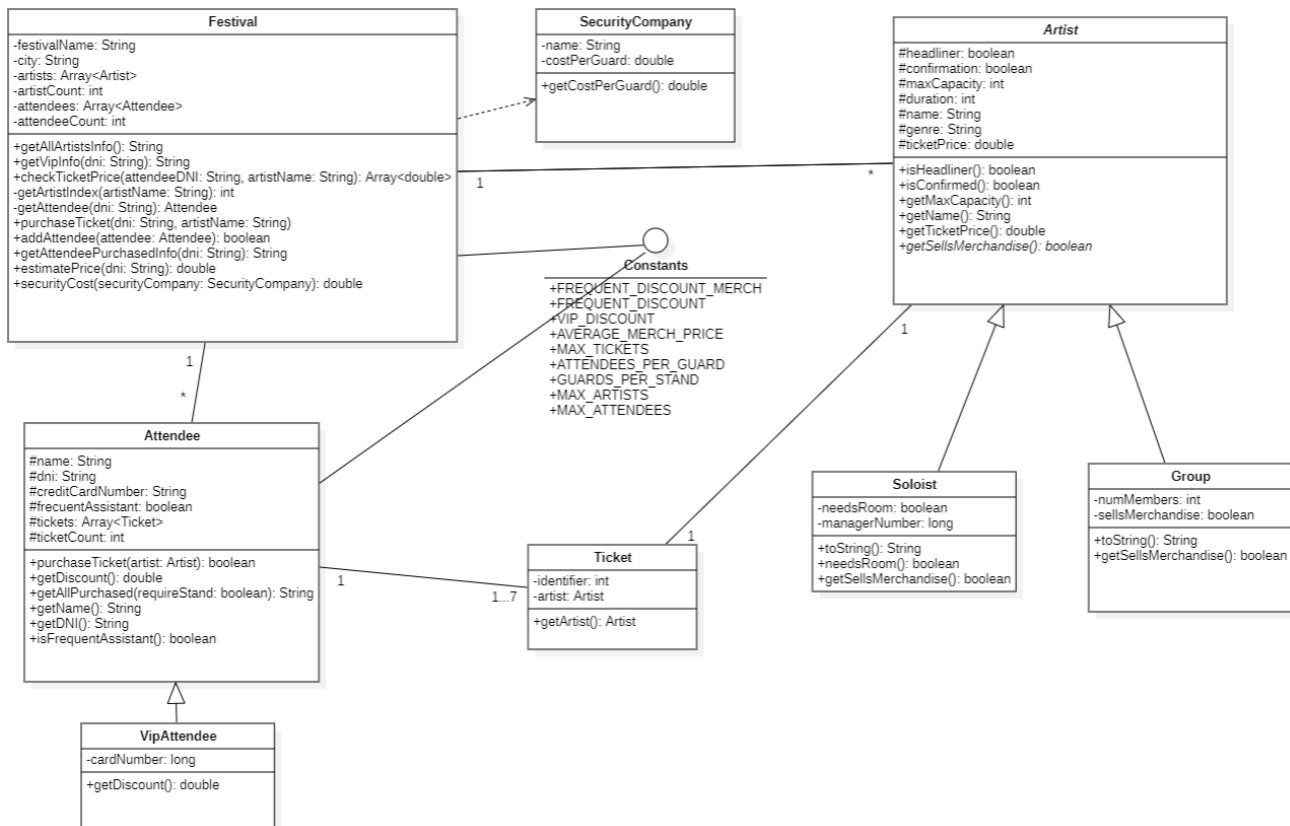
In this project, we have been requested to automate and manage a festival, which is composed of several artists and attendees. On the one hand, artists can be either a group or soloists, and each one of them has its own characteristics and necessities, which will be explained below. On the other hand, attendees can have access to exclusive areas and dressing rooms if they possess a VIP card number; these attendees are called "VIP attendees". Moreover, these attendees must have a ticket to attend the concert, and they cannot have more than 7 tickets. In addition, the festival rents the service of a security company that ensures the fulfilment of the law and guarantee secure conditions during the festival.

With respect to the queries, they are explained below:

- **Query 1:** We need to show the information of all the artists who participate in the festival.
- **Query 2:** We need to calculate the cost of hiring a security company, considering that we need 1 security guard for every 500 attendees, 2 for every merchandise stand, and 1 for every soloist private room.
- **Query 3:** We need to allow attendees to check the cost of a ticket for a concert and calculate the available discounts. Finally, we will show a final message containing the discount applied and the final price.
- **Query 4:** We need to calculate the amount of money it will cost for an attendee to buy a t-shirt at each confirmed headliner concert and go to all confirmed headliner concert.
- **Query 5:** We need to allow an attendee to purchase a ticket. If the attendee is not registered, then their information should be requested.
- **Query 6:** We need show all the information of each concert an attendee has purchased a ticket for.
- **Query 7:** We need to show all the information of the concerts with merchandise stands a VIP attendee will attend.

UML Diagrams:

Below is shown the whole UML diagram that models the project:

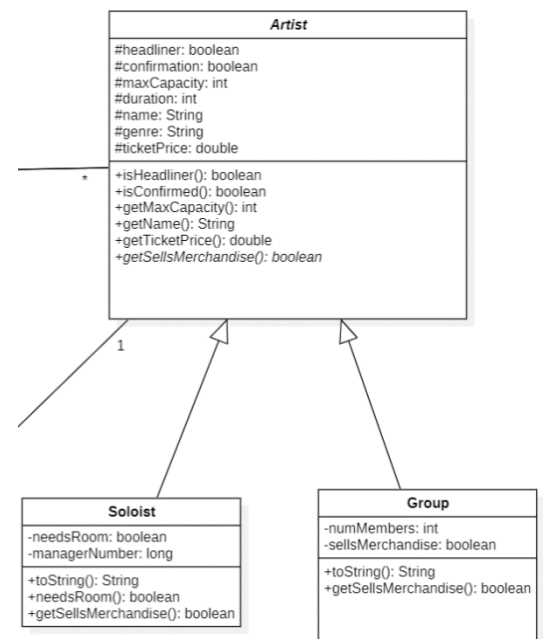


Moreover, we will analyse each class carefully, explaining each one of the relationships they present, as well as their respective attributes and methods.

Artist, Soloist and Group:

Firstly, the Artist class is an abstract class, as well as a father of the Soloist and Group classes. The attributes of this first class include: whether the artist appears as a headliner, whether they have confirmed their assistance, the maximum capacity for their concert in thousands of people, the duration of the concert in minutes, their name, the genre of songs they play and the price charged for 1 ticket. With respect to the methods, all of them are getters, which are used in other classes, such as Festival and Ticket. We should remark that the class Artist has an association relationship with these 2 classes. Finally, the abstract method *getSellsMerchandise()* is implemented in the child classes: since soloists do not sell merchandise, the returned value is *false*.

Secondly, the Soloist class is a child class of the Artist class (as previously mentioned). Its attributes include: whether they need a room and their manager number, as well as the attributes of the father class. In addition, the implemented methods include: the father class methods, a

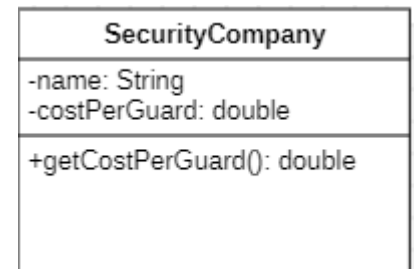


method `toString()` to return the information of the soloist, and 2 getters. The first one is the method `needsRoom()`, which returns whether the soloist needs a room, while the second one is the overridden method of the father class.

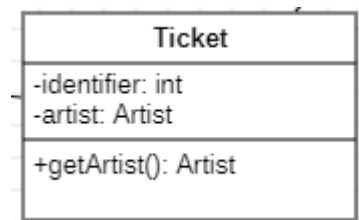
Finally, the Group class is also a child class of the Artist class. Its attributes include: the number of members that composes the group, as well as whether the group sells merchandise. Moreover, the implemented methods include: methods inherited from the father class, the method `toString()` that returns the information of the Group, and the overridden method from the father class. In this case, the attribute `sellsMerchandise` does not necessarily need to be true, since a group can opt for not selling merchandise.

SecurityCompany, Ticket, Constants (Interface):

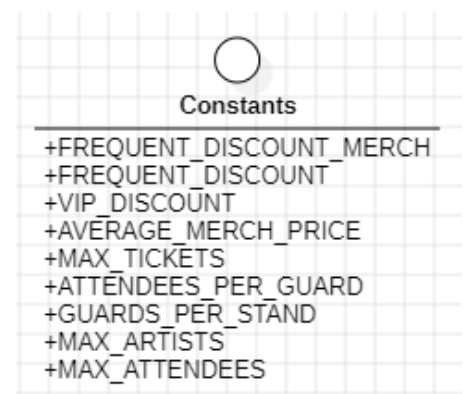
Firstly, the SecurityCompany class is a class that provides a price for each of the guards. It is remarkable that this class is part of a dependency relationship with the class Festival. Its attributes include: the name of the security company, as well as the amount of money charged for each guard. It has only one method, which returns the cost in euros of each guard.



Secondly, the Ticket class represents the ticket Attendees purchase when they are willing to attend to a concert of an Artist. Its attribute consists of an identifier and the Artist it is attributed to. With respect to the method implemented, `getArtist()` returns an instance of the class Artist. We should remark that this class has an association relationship with the classes Attendee and Artist.



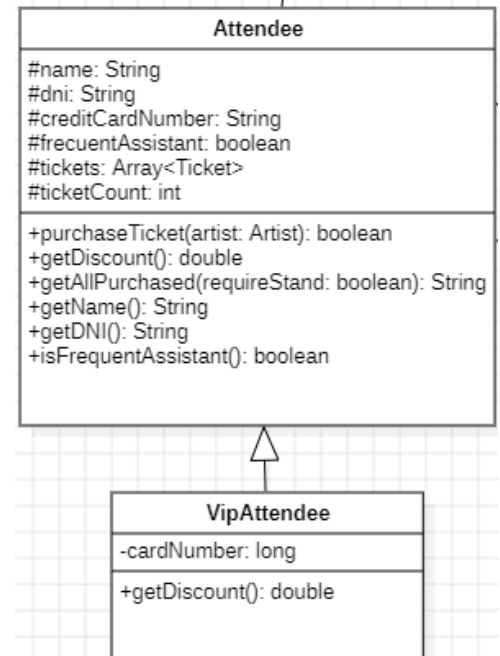
Lastly, the Interface Constants has only attributes, which are used in the Attendee and Festival classes. Among these attributes we can find: discounts, the average price of merchandising, the maximum number of tickets an attendee can have, the proportion of attendees with respect to guards, as well as the number of guards per stand, and the maximum number of artists and attendees.



Attendee, VipAttendee:

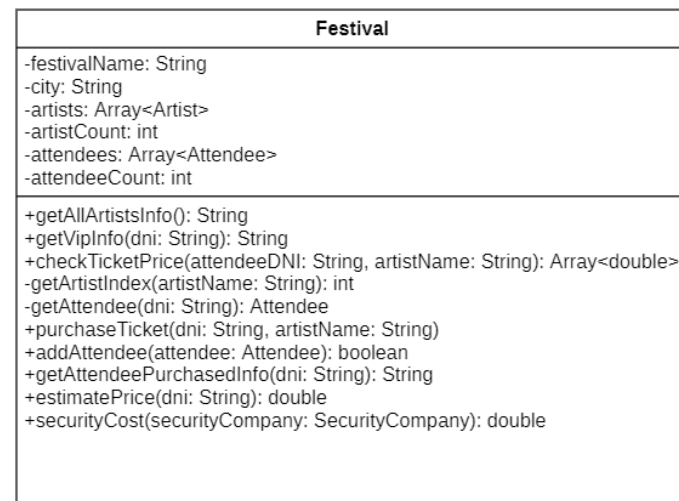
Firstly, the class Attendee is a father class of the VipAttendee class. Its purpose is to model the behaviour of a person that wants to attend to a concert. We can outline that this class implements the interface Constants, and has an association relationship with the Festival and Ticket classes. Among its attributes we can find: its name, dni, creditCardNumber, whether they have previously assisted, all their tickets and the number of tickets they have. Note: In StarUML, it is not allowed to write Ticket[], so to fix this we have opted for writing "Array<Ticket>". This should not be confused with ArrayList<Ticket>. The methods implemented include: a method used to purchase a ticket for an specific concert, and methods for getting the value of some of the class attributes.

Secondly, the class VipAttendee is a child class of Attendee class (as previously mentioned). Its attributes include the ones inherited from the father class, and the number of the VIP card. In addition, the methods implemented are the ones inherited from the father method, as well as a getter method that returns the discount applied for VIP attendees.



Festival:

The class Festival is the most important class of this project, since it encompasses the rest of classes. It is remarkable that this class implements the interface Constants, has a dependency relationship with the SecurityCompany class, and has an association relationship with the classes Artist and Attendee. Its attributes include: its name, city, all the artists and attendees, as well as the number of artists and attendees. With respect to the methods implemented, the two private methods (getArtistIndex() and getAttendee()) are support methods, while the public methods are used to satisfy the queries explained in the first section of the present document. Below is explained which method is used in its corresponding query:



- Query 1: getAllArtistsInfo()
- Query 2: securityCost(securityCompany: SecurityCompany)
- Query 3: checkTicketPrice(attendeeDNI: String, artistName: String)
- Query 4: estimatePrice(dni: String)
- Query 5: purchaseTicket(dni: String, artistName: String)
- Query 6: getAttendeePurchasedInfo(dni: String)
- Query 7: getVipInfo(dni: String)

User Manual:

The source folder is composed of .java files and two .text files. All the files should remain unchanged, so that the program will function correctly. To start the program, you should open the 'GA2_RomeroLoro.java' and execute it in an IDE (Integrated Development Environment).

When we start the program, a menu appears with 8 options. These options provide functionalities within the festival and are available for the user to explore. Notice that the command line requests for a number, which is used to select the corresponding option. Please bear in mind that the user should introduce a number between 1 and 8, otherwise you will be requested again to enter an option.

This program has several artists and attendees, whose information is read from the text files "Artistas.txt" and "Asistentes.txt". These files should not be erased or edited, i.e., changing the name of the file, or adding more information in an incorrect format.

When the user selects an option, one must encounter that the command line requests for the DNI (identification number) of an attendee. In the file 'Asistentes.txt', all DNIs are available, as well as other attributes aforementioned.